Machine Learning Project

Malik Jensen
IT-University of Copenhagen
majen@itu.dk

Matthew Nañaga Rangstrup
IT-University of Copenhagen
mnra@itu.dk

Taha Zaman Khan Khattak
IT-University of Copenhagen
takh@itu.dk

Course Title: Machine Learning **Course Code:** BSMALEA1KU

1 Introduction

This paper is a deep dive into predicting claims risk within the motor insurance industry. We aim to apply various machine learning methods in order to predict claims risk.

We were provided with a dataset over motor insurance claims, with $\sim$ 680000 entries, that was split into training and test data consisting of $\sim$544000 data points and $\sim$136000 data points respectively [1].

The dataset contains the following information:

Name	Description
IDPol	The unique identifier of a given policy
ClaimNb	The number of claims during the exposure period
Exposure	The amount of time at risk in years, with a max of 1 year
VehBrand	Anonymized vehicle brand denoted by B1, B2 etc.
VehGas	Fuel type of insured car
VehPower	Power rating of vehicle, where higher value means higher power
VehAge	Age of vehicle in years
Area	Density class of the community where insured person resides, ranges from A: most rural to F: least rural
Density	Number of inhabitants per square-kilometer where insured person resides.
Region	The administrative region where insured person resides
BonusMalus	Bonus/malus rating, where values $<$ 100 indicates a good record (bonus) and values $>$ 100 indicates a bad record (malus)

Table 1: Truncated version of table explaining different features in the dataset [1]

1.1 Defining Claims Risk

Claims risk is a broad topic, and there are many ways in which one can define claims risk. When defining claims risk for our dataset, it was important to consider which features (if any) would make a good candidate for claims risk, and which variable(s) should become the target variable(s) for the models.

Through research we found various papers dealing with insurance risk, several of which dealt with either number of claims or predicting insurance premium costs. [2] [3]

Since we do not have access to any information about the price of the insurance claims, we decided to move forward with the claims number as the measure of claims risk for this project. The frequency of claims was also considered, where claim count would be divided by the amount of exposure, but this was difficult given that a data point of 0.1 exposure and 0 claims is not the same as a data point with 1 exposure and 0 claims.

We also considered using bonus-malus as a risk measure, but due to bonus-malus being based on historical data [4], we decided that number of claims was a better fit for the task at hand, due to a lack of long-standing historical data within the dataset.

2 Data Cleaning & Processing

The following section will be covering our data cleaning methodology.

2.1 Dropped Features

One of the first things discussed are the different features, and whether or not any of them should get dropped. IDPol, is deemed insignificant, as the official description of the project states that IDPol is defined as: *"The ID of the Policy, which is a unique identifier representing the contract of a person and their car to the insurer."*[1] Hence it does not tell us anything useful in predicting claims.

The description of the area feature was almost identical to the description of the density feature, with area being defined as: *"The area (or **density class**) of the city / community where the car driver lives. This is an ordinal label starting from "A", where "A" is most rural, "B" is less rural, and so on."* [1] and density being defined as: *"The number of inhabitants per square-kilometer in the location where the driver lives (i.e. **population density**)."* [1]. Since they both mention density in their description, we decided to look into whether or not there was any correlation between the two.

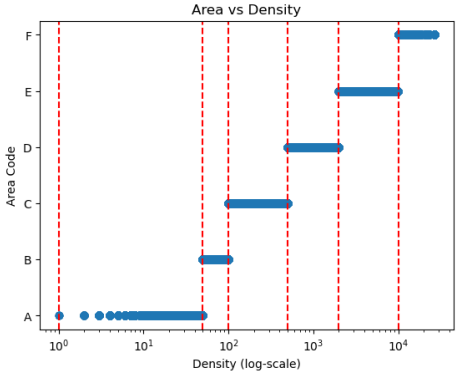

Figure 1: Area plotted against density (log-scale)

By plotting out area as a function of density, we discovered that there is a 1-to-1 relationship between the area and density.

We removed the area feature, as it is not very precise for certain values of density, such as area F, which spans from a density of about 10000 to a value of 27000.

2.2 Outlier Detection

A major part of our data cleaning process was spent on trying to find outliers within different features of the dataset. In the following section the most important findings of the data cleaning part will be shown.

2.2.1 Exposure

Exposure tells us the length of the policy, and within the exposure column we found values that went above 1, meanwhile the information provided mentioned that *"The dataset for the project contains an overview of vehicular insurance policies and their claims history* ***over the last year***."[1]

Therefore we ended up removing all data points with exposure greater than 1.

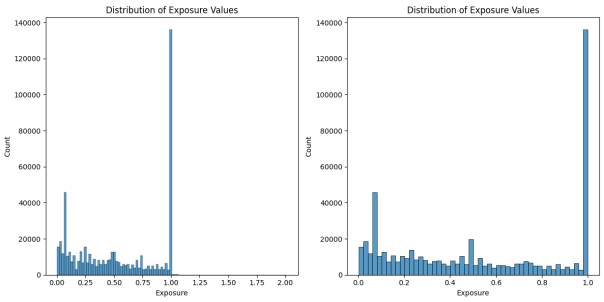

Figure 2: Left: Histogram over Exposure in training set before removal of outliers
Right: Histogram over Exposure in training set after removal of outliers

2.2.2 Vehicle Age

Through visual inspection of the vehicle age feature, we see that, past 40 years of age, there are barely any cars in the dataset, although it ranges all the way up to 100.

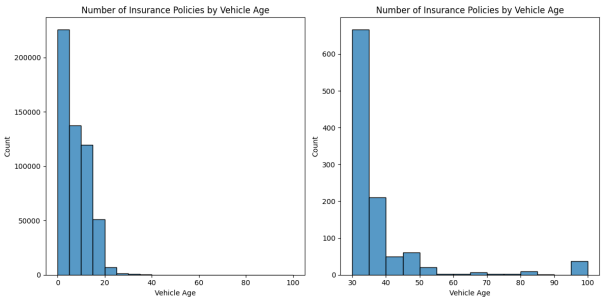

Figure 3: Left: Number of insurance policies by vehicle age over all vehicle ages (0-100)
Right: Number of insurance policies by vehicle age for vehicle ages ≥ 30

Focusing specifically on vehicle age values at 30 years and above, we see a sharp drop-off after 40 years. We see that there is only $\sim$200 cars in the range from 40-100. Due to the low amount of cars in that range, we decided to treat these as outliers and removed them accordingly, in order to not mess up the scaling of our vehicle age feature.

2.2.3 Driver Age

Through the visual inspection of the feature driver age we also spotted anomalies, especially in the high ranges above 85 years old.

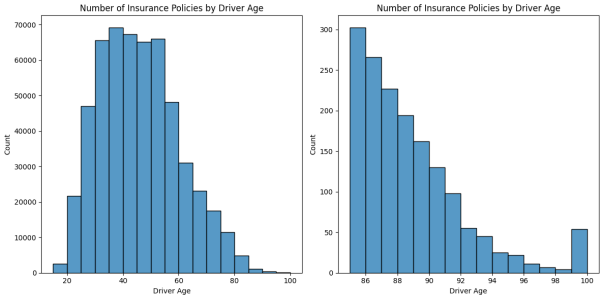

Figure 4: Left: Number of insurance policies by driver ages over all driver ages (18-100)
Right: Number of insurance policies by driver ages $\geq$ 85

While the impact on variance and scaling is not as bad in this instance compared to vehicle age, we still decided to drop the data points of driver ages that went above 92 years old, as the sample size for each age started rapidly declining from that point, with the exception of 100 where it increased slightly again.

2.2.4 Bonus-malus

From visual inspection of the bonus-malus feature, we also see a severe lack of data points past the bonus-malus score of 130. Closer inspection reveals that there seems to be outliers past a bonus-malus score of 160.

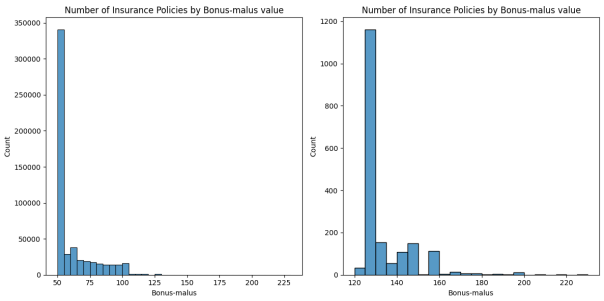

Figure 5: Left: Number of insurance policies by bonus-malus score over all scores (50-230)
Right: Number of insurance policies by bonus-malus score ≥ 120

Here we decided to remove the data points with bonus-malus scores above 160, due to a lack of data points, since these could interfere with both principal component analysis and scaling algorithms due to high variance.

2.3 Scaling

In order to perform dimensionality reduction using Principal Component Analysis (PCA), and for certain models when training, such as neural networks, it is important that numerical data is scaled properly, due to these methods relying on variance or distance based measures, which are highly sensitive to the scale of the data itself. [5]

Scaling is not a replacement for outlier detection, since extreme outliers will still affect scaling processes, it is simply a way to reduce the values within each predictor itself, such that one predictor is not considered more important in an algorithm such as PCA, simply due to having greater values.

When deciding on a method of feature scaling we ended up using StandardScaler [6]. While there are other methods, such as MinMaxScaler, and RobustScaler, it is generally understood that StandardScaler works well for most datasets and algorithms [7].

When transforming the data using the standard scaler, we made sure to fit the scaler on the training data and then apply it on the test data for each feature respectively, in order for the features in both our training and test data to follow the same distribution, such that our data is scaled relative to the training data. Otherwise we would end up with two different scales, which would not work for predictions.

3 Data Visualization

In order to visualize the data, we made use of two different methods, namely Principal Component Analysis (PCA) and K-Means clustering.

3.1 Principal Component Analysis

When doing PCA, we decided to use it for all of the quantitative variables. While it would have been possible to one-hot encode the categorical variables and include them in the principal components, it is normally recommended to use them strictly for quantitative variables [8]. This meant we ended up with six different principal components, due to having six numerical features.

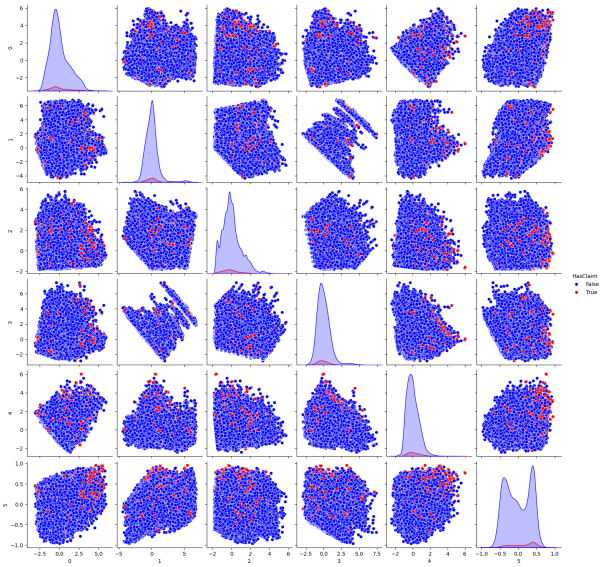

Figure 6: Scatter plot over the six principal components

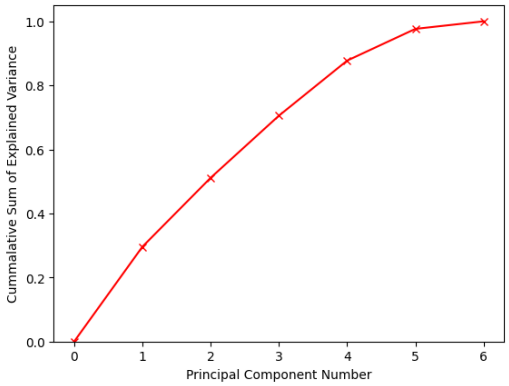

Figure 7: Cumulative plot over amount of explained variance by number of principal components

When looking at the amount of explained variance, we see that the first four principal components describe about 90% of the variance, and then the amount it increases by drops off sharply for the last two components. While this method was interesting to take a look at, we decided to move forward with the initial features, since we are not affected by the curse of dimensionality in terms of the numerical features.

3.2 K-Means Clustering

For k-means clustering, we used the elbow method in order to decide on a good value for the amount of clusters k. The elbow method essentially relies on using a metric such as inertia, which is defined as:

$$\text{Inertia} = \sum_{i=0}^{n} \underset{\mu_j \in C}{min}(||x_i - \mu_j||^2) \tag{1}$$

This is essentially just the within-cluster sum of squares [9]

With the elbow method we then look at where the resulting graph has a visual bend, similar to how an elbow bends [10]. This led to the following results.

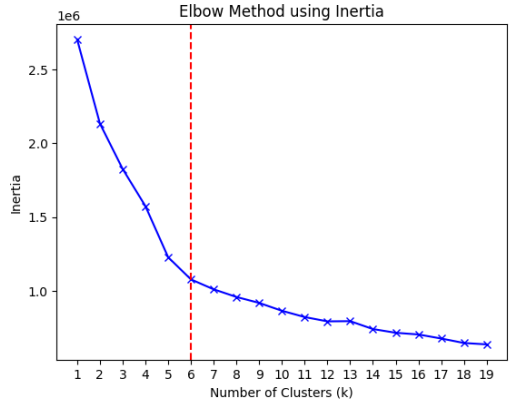

Figure 8: Plotting inertia value of each k for the elbow method

From this, we saw a clear bend at 6 clusters, and hence we decided to use 6 clusters to create the visualization of the data.

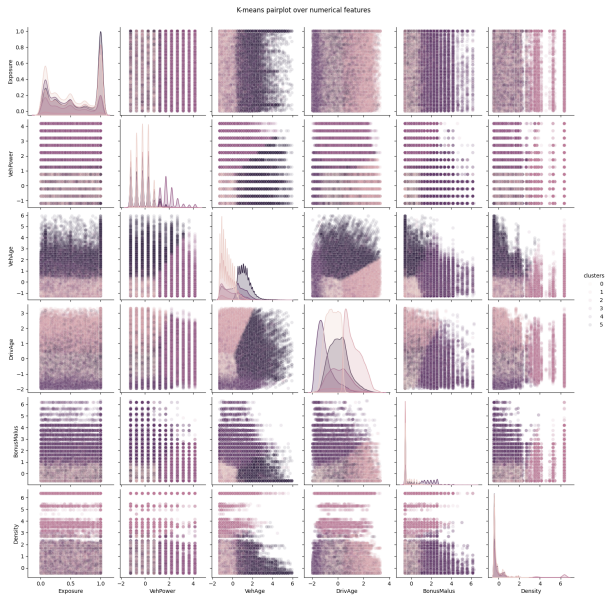

Figure 9: Pairplot over K-Means clustering with 6 clusters and alpha $= 0.1$, less transparent areas have more points clustered together.

The reason that most plots look like stripplots, is due to most of the numerical variables being discrete, such as VehiclePower and VehicleAge.

4 Modelling

As per project requirements, we are asked to explore two different methods, decision trees, and neural networks. From there, we are asked to develop and explore our own third method, informed by our previous results.

4.1 Loss Function

Since we are dealing with number of claims, it is important to end up with predictions that are only 0 or larger, and we also wish to make use of a discrete distribution which is great for counts, due to dealing with discrete numbers.

This is where typical metrics such as residual sum of squares does not perform as well, as the model punishment always stays the same when equally far away.

In the instance of claims data, it is important to punish certain scenarios more, such as predicting claims when making no claims, due to number of estimated claims tying directly into their risk profile, hence the premium the customer ends up paying increases.

We can use the Poisson distribution as a loss function in order to map the number of claims, where the exposure for a given sample is used as a sample weight [11][12]. This can be achieved through the mean Poisson deviance loss given by

$$\begin{aligned} &\text{for } y_i > 0: \\ &\quad \frac{1}{n}\sum_{i=1}^{n} w_i \cdot 2(y_i \log(y_i/\hat{y}_i) + \hat{y}_i - y_i) \\ &\text{for } y_i = 0: \\ &\quad \frac{1}{n}\sum_{i=1}^{n} w_i \cdot 2(\hat{y}_i) \end{aligned} \tag{2}$$

Where:

- n is the sample size
- w_i is the sample weight for sample i
- y_i is the true value for sample i
- $\hat{y}_i$ is the predicted value for sample i
- log is the natural logarithm

The underlying Poisson distribution makes the assumption that the variance and mean is equal, while this particular detail has not been verified in this

dataset, variations of the Poisson deviance loss function has been used in other papers dealing with predicting number of claims, and as such, we decided to move forward with this idea. [13]

4.2 Baseline

For a baseline comparison to our models, we create a baseline model. It simply gets the mean of all the training claim numbers, and predicts that for every single value. This helps us compare neural networks that may simply predict a value around 0 and 1 in order to minimize losses, due to the high number of zero-claims.

This results in a loss of around 0.31897, with no variance amongst the predictions. The mean prediction is around 0.0531, which is obvious, given the inflated number of zero-claims in the data.

4.3 M1: Decision Tree Regressor

The first model we implemented was the decision tree regressor, with 3 hyperparameters, namely MaxDepth, MinSampleSplit and MaxLeafs. The scratch implementation recursively makes the best split out of all possible splits, one which minimizes a loss function, which in our case is either the mean Poisson deviance loss (4.1) or the residual sum of squares (RSS).

$$RSS = \sum_{i=1}^{n} (y_i - \bar{y}_i)^2 \tag{3}$$

Where:

- n is the sample size
- y_i is the ith data point
- $\bar{y}_i$ is the mean of the claims in a leaf node, as well as the prediction for that node.

The recursion stops when a stopping condition is reached. Then once the tree is built, we can traverse the tree for a given data point and make a prediction based on it.

4.3.1 Model Selection

Once the implementation was complete, we confirmed that the tree behaves similarly to the scikit-learn implementation. It was time to decide which values the hyperparameters should use.

The hyperparameters which needed to be fit were the maximum depth of the tree, minimum samples for a split and the maximum number of leaves.

To decide on the best combination of these hyperparameters we had two options, either using GridSearchCV, which looks at all possible combinations of the hyper-parameters, or use RandomizedSearchCV, which samples random values, from a range of given values, to find the best combination. [14] One can already see the clear advantage of RandomizedSearchCV in terms of efficiency, as it only tests a subset of the parameters, rather than every possible combination. RandomizedSearchCV is often preferred in larger datasets like ours.

RandomizedSearchCV also implements cross validation, such that rather than training on the entire training set, it creates K folds (splits), in each iteration trains on K-1 folds and tests on the remaining fold. This makes the training results more varied rather than always fitting on the same data, and it also allows us to tune hyperparameters without ever seeing the held out test data. [15]

We ran RandomizedSearchCV on our implementation, with 100 iterations and K=5 for Kfold. We got the following optimal values for the hyperparameters on the training set.

$$\begin{aligned} \textit{Max Depth} &= 5 \\ \textit{Max Leaf Nodes} &= 29 \\ \textit{Min Sample Splits} &= 70 \end{aligned} \tag{4}$$

It is important to mention here, that for the sake of efficiency, we amended our implementation of the decision tree, such that instead of looking at all possible splits in the best_split method, we only considered 100 candidate splits which were derived from the percentiles of that features distribution. We also made it such that it inherits from scikit-learn's BaseEstimator and RegressorMixin, which are methods vital to RandomizedSearchCV's function.

Another choice made was the inclusion of certain categorical variables. We chose only one categorical variable, VehGas, in our final decision tree model. We found that regardless of the inclusion of the rest of the categorical variables, we arrived at similar results. VehGas is also a binary variable, which allows us to keep the dimensions as low as possible.

4.3.2 Evaluation of Model

Once we had the optimal hyper-parameters, we needed another measure of the quality of our fit, instead of just of minimizing our loss function. Our first intuition was to use R^2 as it allows us to compare between models. We quickly realize, however, that R^2 is a measure based on MSE, while we are focused mainly on mean Poisson deviance. As we know one of the assumptions MSE makes is that of homoscedasticity, however our loss function does not make that assumption.[16]

While looking for alternatives to R^2 we came across the R^2 equivalent when it comes to the

Tweedie distributions, the family of distribution used to derive the mean Poisson deviance. This was the D^2 Tweedie score. [17]

$$D^2 = 1 - \frac{PoissonDeviance(Model)}{PoissonDeviance(Null)} \quad (5)$$

The D^2 takes the ratio of how much Poisson deviance is explained by our model compared to a null model which just predicts the mean.

On the training data, using k-fold, and the optimal hyper-parameters, we get a D^2 value of 0.065 (6.5%). This is not great as it means only 6.5% of the variance was explained in the training data.

4.4 M2: Feed-Forward Neural Network Regressor

The second model we implemented was the neural network regressor. It has 38 nodes in the input layer (including exposure and after converting categorical values into dummy variables) and 1 node in the output layer.

4.4.1 From-scratch Implementation

Using code adapted from a provided exercise, we implement our own from-scratch implementation. Using 32 nodes in the hidden layer, we train a neural network on the data. Because of the rudimentary implementation, a lot of hyper-parameters are fixed, such as one hidden layer, and a batch size equal to the size of the data, equivalent to batch gradient descent. Another such case is the gradient descent optimization, of which there is none.

There are a variety of activation functions that can be used in the hidden layers, but the two that were considered are ELU and ReLU.

An issue that arose often when trying to calculate mean Poisson deviance, was that predictions may result in not strictly-positive values, because the initialization of weights would be random and could result in negative values. Mean Poisson deviance requires that predicted values are strictly positive. This can be fixed through a log-link function.

Two options, that were considered, included the Softplus function and the exponential function.

Log-link Function and Exposure Offset When modeling count data, like insurance claims, which tend to form Poisson distributions, we can always ensure our prediction is a positive value using a log-link function. That is to say:

$$\hat{y} = e^{\beta_X} \quad \text{or} \quad \log \hat{y} = \beta_X$$

Where $\hat{y}$ is our predicted value, and β_X is the outputted value from our model, given input X, that corresponds to the prediction $\hat{y}$. The log or exponential function is a transformation of the model's output into a strictly positive value for our prediction. [18]

It should also be considered that someone who has no claims and exposure of 1 is different in terms of risk to someone who has an exposure of 0.1. There is large risk for the insured with lower exposure to file more claims within a 0.9 year period. We can instead model the claim rate as so:

$$\frac{\hat{y}}{t} = e^{\beta_X}$$

We can transform this equation such that:

$$\hat{y} = t \cdot e^{\beta_X} \quad \text{or} \quad \log \hat{y} = \beta_X + \log t$$

We are still predicting claim counts in a sense, but with the added information of exposure, which 'punishes' the output of the model with lower values of t, telling us that there is inherently more risk that the model should account for to get a closer prediction. [19]

Such a scratch implementation was attempted, resulting in one of the worst results of all the models tried, performing even worse than the baseline. With ELU as the hidden activation function, exponential function as the output activation, batch gradient descent, 32 hidden nodes, $\alpha = 0.05$ learning rate, and 300 max epochs, we end with a validation loss of 0.325, but notably, our highest prediction variance, of 0.00296 claims.

We realize that our neural network is likely to learn the nuances between prediction and exposure, regardless of exposure offset (of which may be more useful in other models). Our next models instead ditch offset exposure in favor of using exposure as a feature. Without offset exposure, we only have difference in the output layer activation function for the next two models, either the exponential or the Softplus function. With the exponential function, we get a validation loss of 0.3121. Conversely, Softplus garners a validation loss of 0.3114. There is a clear improvement, but nothing significantly different between the two. There is respectively 0.000614 and 0.000652 variance.

Up until this point, the output activation function was the log-link function, but we can instead abstract the process, where the output layer activation function is linear, which then is transformed by the log-link function, having no involvement in the gradient calculation in the neural networks. Using the exponential and Softplus functions once again, we get 0.3042

and 0.3038 validation loss. There is improvement, but nothing significant between the two. Swapping our hidden layer activation function to ReLU, achieves no significant difference either, a value of 0.3039. There is no notable significant difference in variance, up to ≈ 0.001.

Finally, changing the number of nodes in the hidden layer, sticking with the exponential log-link function, and ReLU, from 32, to 64, to 256, gives us 0.303 and 0.3028 respectively, showing apparently small diminishing returns, but obvious improvements given that more hidden nodes may easier learn patterns in the data. We continue forward with 32 hidden nodes due to the relatively small improvements we gain, to the detriment of runtime.

At this point, our neural network seemed to be pushed as far as we could take it with our abilities and a library reference implementation with the PyTorch library, using 32 hidden layers, ReLU/Linear activation, and exponential log-linking, and stochastic gradient descent, gives us a validation loss of 0.3353885891.

When using the Adam optimizer, we get a loss of 0.3007. The only effect the difference in optimizers give us is the speed at which we arrive at the minimum, which should not affect our results significantly, given we ran training for a large number of epochs. The closeness of the losses, given random noise as well, gives us good reason to believe that our from-scratch implementation is quite correct.

4.5 M3: Ensembles

Our final method involved trying to combine our models in some way. Here, we look toward ensembles. Ensembles are a group of predictors, that we take predictions from, and aggregate into a final prediction value, as their combined guesswork often results in better values.

4.5.1 Boosting

Boosting is an ensemble method that uses several weak learners in sequence, trying to make up for the errors that its predecessor makes. In particular, we used gradient boosting, which tries to fit the new predictor based on the residual errors made by its predecessor [20]. To find the best hyper-parameters, we use RandomizedSearchCV.

Results Unoptimized: Performs so poorly, it never guesses a positive-claimant correct and thus has no precision nor recall.

$$\text{Accuracy: } 0.9497\ (94.97\%)$$

Optimized hyper-parameters:

$$\text{Accuracy: } 0.9492\ (94.92\%)$$
$$\text{Precision: } 0.3191\ (31.91\%)$$
$$\text{Recall: } 0.002757\ (0.2757\%)$$

4.5.2 Stacking

Stacking is when we train a separate model to "blend" together predictions of multiple other models. This is in comparison to something like a hard-voting classifier that would simply take a majority vote from every model [20]. The idea is to train multiple of our best models that are all weak learners into one strong learner, taking the strengths of each.

We first create a simple stacker with 4 models within:

- Our 'best' model from M2, neural networks.
- Our gradient boosting classifier.
- A support vector machine (SVM) classifier (SVC), which essentially attempts to find a hyperplane in feature space that best splits the data between our classes, creating the largest 'street' [20].
- A random forest classifier, which is another ensemble, that essentially takes the mean of multiple decision trees that are all individually trained on smaller subsets of the training data [20].

First we attempt a stacking classifier with these predictors, with most of their library-default hyper-parameters, then we attempt to make a stacking classifier, after finding the best hyper-parameters for each predictor individually, to then combine into a stacking classifier. Finally, we try a stacking classifier, where we try to find all hyper-parameters simultaneously. To find the best hyper-parameters, we once again use RandomizedSearchCV.

Results Our 'unoptimized' stacking classifier got:

$$\text{Accuracy: } 0.9492\ (94.92\%)$$
$$\text{Precision: } 0.4421\ (44.21\%)$$
$$\text{Recall: } 0.02117\ (2.117\%)$$

The 'partially-optimized' stacking classifier has:

- Random forest classifier with 200 trees, and max depth 20.
- Our neural network with 256 hidden nodes, ReLU, Linear Activation, learning rate 0.05.
- A Linear SVC with L2 (ridge) regularization and regularization parameter 0.75.

- Our gradient boosting classifier with learning rate $\alpha = 0.46$ and max depth 7.

$$\text{Accuracy: } 0.9494 \ (94.94\%)$$
$$\text{Precision: } 0.3021 \ (30.21\%)$$
$$\text{Recall: } 0.005331 \ (0.5331\%)$$

Shockingly, we got similar accuracy but worse precision and much less recall.

The 'fully-optimized' stacking classifier has:

- Random forest classifier with 200 trees, max depth 20, bootstrapping and out-of-bag evaluation, 4 minimum samples per split, where max features is $\log_2(n)$.
- Our neural network with 192, 48, and 192 hidden nodes in layers, ReLU, Linear Activation, batch size 1000, learning rate 0.0001.
- A Linear SVC with L2 (ridge) regularization and regularization parameter 0.75, early stopping tolerance 1E-06, and hinge loss.
- Our gradient boosting classifier with learning rate $\alpha = 0.2$ and max depth 20, max features 0.75.

$$\text{Accuracy: } 0.9492 \ (94.92\%)$$
$$\text{Precision: } 0.3893 \ (30.21\%)$$
$$\text{Recall: } 0.008468 \ (0.8468\%)$$

Which once again, gets similar accuracy but worse precision and much less recall, but better than our 'partially-optimized' classifier.

5 Results

5.1 Decision Tree

For the final test of the decision tree, we trained the tree using the optimal hyper-parameters obtained through RandomizedSearchCV, on the full training data set. We then used that tree to predict claim values for the held out test set. These were the results obtained;

$$D^2 = 0.065 \ (6.5\%)$$

$$\text{Mean Poisson Deviance} = 0.302$$

5.2 Neural Network

Evaluating our model from our from-scratch implementation on the held-out test data, trained on the optimal hyper-parameters found through the small search performed, we get the results:

$$D^2 = 0.0482 \ (4.82\%)$$

$$\text{Mean Poisson Deviance} = 0.3084$$

Notably, there is a 0.000906 variance in the predictions.

Figures 10 and 11 compare predicted values with the true values from the held-out test set for the optimal M1 and M2 scratch-implementation.

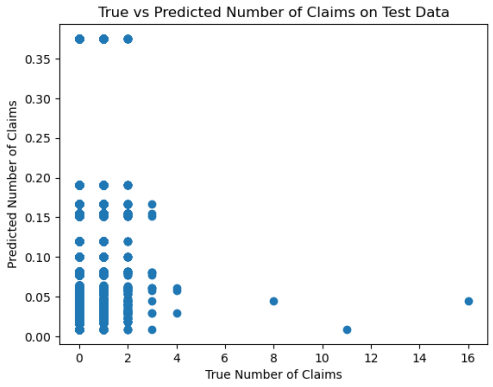

Figure 10: Actual (X) vs Predicted (Y) Values for Test Data (Decision Tree)

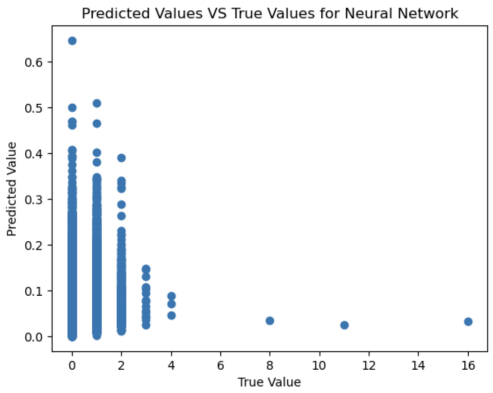

Figure 11: Actual (X) vs Predicted (Y) Values for Test Data (Neural Network)

5.3 Boosting and Stacking

Evaluating our final stacked model on the held-out test data, we get:

$$\text{Accuracy: } 0.9492 \ (94.92\%)$$
$$\text{Precision: } 0.3893 \ (30.21\%)$$
$$\text{Recall: } 0.008468 \ (0.8468\%)$$

6 Discussion

We believe a core issue with the poor performance of our models was due to the highly imbalanced dataset. 95% of the insurances had 0 claims in the training set. This makes it very hard for most models to learn patterns from. This can be clearly seen in Figure 10 where the max prediction for any claim is just above 0.35, whereas the maximum number of claims in the test set are 16 claims. Most of the data points contain 2 or less claims.

One way we sought to improve our models was by turning this into a classification problem rather than a regression problem for our M3 method. We hypothesized that attempting to slightly balance the dataset, given the mass amount of no-claims may lead to more meaningful results. However, we lose the information that we gain from continuous values of claims.

Our regressors may tend to predict higher claims for higher true claim values, even if we never predict higher than 1 claim. This could itself be interpreted as a measure of risk. In particular, one can see a trend in the minimum values of Figure 11, where higher true values mean higher minimum values in the range of the predictions.

On top of this, we could have tried to improve model performance through some sort of resampling. An example of this could be stratified sampling for our classification model, where we can choose to over-sample the insurances which have claims, and under-sample those without any claims. Even though this would result in loss of data and not be reflective of the true proportion of our data, it might help the model in finding useful patterns and not being biased towards the majority class.

A re-occurring issue we find is that our models are severely under-fitted, with high bias and low variance. Their predictions always land in a particular area, and rarely stray.

The removal of area as a feature, instead of density, was a good choice, as the use of area could be seen as a form of regularization. An increase of bias in favor of a decrease in variance, by stripping the information gained by the differences between data points in an area group. This is the opposite of what we want to improve our models (trading higher variance for lower bias). [21]

7 Conclusion

Between all models, we tend to prefer those with higher precision and recall in insurance claim modeling. From the perspective of an insurance company, it is more valuable that we are alerted of insurers who are risky, so that we may adequately price them. Therefore, we wish to reduce the error performed in our raised flags (as in precision), and wish to fully encompass risky individuals in our raised flags (as in recall).

Deep decision trees are typically high variance, low bias, but can decrease both bias and variance through bagging and aggregation, like in random forests, or less variance in boosting classifiers. Neural networks are typically high variance, low bias, as they tend to learn complex patterns, especially as more hidden layers and nodes are added.

Despite both of these realities, both methods still seem to severely under-fit the data, where they tend to guess in a small range of possible values or too often guess no-claimants. For regression, our models often imitate a baseline model, which is one of the most under-fitted models, given their high bias and low variance.

For classification, for example, decision trees often get very pure leaf nodes regardless of their splits. There is simply not enough features nor data for any method to intimately learn the patterns in the data well enough to predict claims effectively.

With problems like these, the main issue comes down to the bias-variance trade-off. We find we have a very underfitting, biased set of models, and we wish to trade off lower bias for higher variance, but it isn't possible given the dataset provided.

References

[1] Jonas Juul. *Machine Learning Exam Assignment*.

[2] V. Selvakumar et al. "Predictive Modeling of Insurance Claims Using Machine Learning Approach for Different Types of Motor Vehicles". In: *Universal Journal of Accounting and Finance* 9.1 (Feb. 2021), pp. 1–14. ISSN: 2331-9712, 2331-9720. DOI: 10.13189/ujaf.2021.090101. URL: http://www.hrpub.org/journals/article_info.php?aid=10494 (visited on 11/29/2025).

[3] Lukas Nyström and Oscar Witt. "Predicting Vehicle Insurance Premiums Using Linear Regression, XGBoost, and Neural Networks". In: ().

[4] Jean Lemaire. "Bonus-Malus Systems". In: *Wiley StatsRef: Statistics Reference Online*. Abstract is the only important part here hence linking it, as it quickly explains bonus-malus. John Wiley & Sons, Ltd, 2014. ISBN: 978-1-118-44511-2. DOI: 10.1002/9781118445112.stat04721. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/9781118445112.stat04721 (visited on 11/29/2025).

[5] Muneeb Alam. *Feature Scaling: Boost Accuracy and Model Performance*. URL: https://datasciencedojo.com/blog/feature-scaling/ (visited on 11/27/2025).

[6] *StandardScaler*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html (visited on 11/30/2025).

[7] *Feature Scaling and Normalization — ML Fundamentals*. URL: https://superlinked.com/glossary/feature-scaling-and-normalization (visited on 11/30/2025).

[8] Brandon Walker. *PCA Is Not Feature Selection*. Dec. 2019. URL: https://medium.com/data-science/pca-is-not-feature-selection-3344fb764ae6 (visited on 12/14/2025).

[9] scikit-learn. *2.3. Clustering*. URL: https://scikit-learn.org/stable/modules/clustering.html (visited on 12/15/2025).

[10] *Elbow Method for Optimal Value of k in KMeans*. Nov. 2025. URL: https://www.geeksforgeeks.org/machine-learning/elbow-method-for-optimal-value-of-k-in-kmeans/ (visited on 12/17/2025).

[11] *3.4. Metrics and Scoring: Quantifying the Quality of Predictions*. URL: https://scikit-learn.org/stable/modules/model_evaluation.html#mean-tweedie-deviance (visited on 11/26/2025).

[12] *Mean_poisson_deviance - Scikit-Learn*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_poisson_deviance.html (visited on 11/26/2025).

[13] Alexander Noll, Robert Salzmann, and Mario V. Wuthrich. "Case Study: French Motor Third-Party Liability Claims". In: *SSRN Electronic Journal* (2018). ISSN: 1556-5068. DOI: 10.2139/ssrn.3164764. URL: https://www.ssrn.com/abstract=3164764 (visited on 11/26/2025).

[14] *Comparing Randomized Search and Grid Search for Hyperparameter Estimation in Scikit Learn*. June 2025. URL: https://www.geeksforgeeks.org/machine-learning/comparing-randomized-search-and-grid-search-for-hyperparameter-estimation-in-scikit-learn/ (visited on 12/17/2025).

[15] M. Hammad Hassan. *Tuning Model Hyperparameters With Random Search*. Nov. 2023. URL: https://medium.com/@hammad.ai/tuning-model-hyperparameters-with-random-search-f4c1cc88f528 (visited on 12/17/2025).

[16] *Homoscedasticity in Regression*. July 2025. URL: https://www.geeksforgeeks.org/machine-learning/homoscedasticity-in-regression/ (visited on 12/18/2025).

[17] scikit-learn. *D2_tweedie_score*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.d2_tweedie_score.html (visited on 12/18/2025).

[18] Moses Kurui. *Mastering Log & Inverse Links in Python Statsmodels GLMs*. Sept. 2025. URL: https://codepointtech.com/mastering-log-inverse-links-in-python-statsmodels-glms/ (visited on 12/20/2025).

[19] Ajay Tiwari. *Offsetting the Model — Logic to Implementation*. Apr. 2020. URL: https://medium.com/data-science/offsetting-the-model-logic-to-implementation-7e333bc25798 (visited on 12/20/2025).

[20] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition*. Place of publication not identified: O'Reilly Media, Inc., 2019. ISBN: 978-1-4920-3263-2.

[21] *Stratified Sampling in Machine Learning*. July 2025. URL: https://www.geeksforgeeks.org/machine-learning/stratified-sampling-in-machine-learning/ (visited on 12/19/2025).